LLM-Guided Evolutionary Search: FunSearch and AlphaEvolve

The convergence of large language models and evolutionary computation has produced a new class of search algorithms in which LLMs serve not as end-to-end solvers but as intelligent mutation operators within an evolutionary loop. Rather than prompting a model to output a solution directly, these systems evolve *programs* that encode solutions, leveraging the model's capacity for code generation while relying on automated evaluators to supply rigorous fitness signals. This section traces the development of this paradigm from its conceptual foundations through its most recent extensions, highlighting FunSearch, AlphaEvolve, and several contemporaneous systems that collectively define the frontier of LLM-guided evolutionary program search.

Foundations: Evolution Through Large Models

The theoretical groundwork for using LLMs as evolutionary operators was laid by Lehman et al. [Lehman2024], who observed that language models trained on large corpora of source code and version-control diffs can approximate the kinds of perturbations that human programmers apply when iteratively refining software. In their framework, dubbed Evolution through Large Models (ELM), the LLM replaces the hand-designed mutation and crossover operators of classical genetic programming with *intelligent perturbation*: given a piece of code and an optional natural-language commit message describing the intended change, a diff model proposes semantically meaningful modifications. When combined with MAP-Elites for diversity maintenance, ELM generated hundreds of thousands of functional Python programs producing locomoting robots in the Sodarace domain, a task that fell entirely outside the LLM's pretraining distribution. The work demonstrated that LLMs could serve as open-ended variation operators, but left open the question of whether such systems could produce genuinely novel scientific or mathematical results.

FunSearch: Mathematical Discovery via Program Search

FunSearch, introduced by Romera-Paredes et al. and published in *Nature* in January 2024 [RomeraParedes2024], answered that question affirmatively. The system pairs a pretrained LLM (specifically, a Codey model) with a deterministic evaluator inside an island-based evolutionary loop. Crucially, FunSearch searches in *function space* rather than solution space: instead of asking the LLM to output a cap set or a bin-packing assignment, it asks the model to write a Python function whose execution produces the desired object. This indirection is what makes the approach both scalable and verifiable, since the evaluator can check the output of any candidate program with certainty.

The evolutionary architecture employs an island model to balance exploration and exploitation. The population of programs is distributed across m independent islands, each initialized with a copy of a user-provided seed program. When constructing a prompt, the system uniformly samples an island, then selects $k = 2$ programs from within that island using a hierarchical scheme: programs are first clustered by their *signature* (a vector of scores on several benchmark inputs), and sampling favors clusters with higher scores while preferring shorter programs within each cluster. The two selected programs are arranged in ascending order of quality and presented to the LLM as successive "versions," implicitly encouraging the model to generate a further improvement. Periodically, the lowest-performing islands are reset and re-seeded from higher-performing ones, preventing premature convergence while preserving the diversity that fuels continued exploration.

Applied to the cap set problem in extremal combinatorics, FunSearch discovered constructions of large cap sets that surpassed the best previously known results, including the largest improvement to the asymptotic lower bound in twenty years. In dimension $n = 8$, the system found a larger cap set from scratch, without requiring explicit knowledge of how to combine lower-dimensional solutions, as prior human-designed constructions had done. FunSearch also demonstrated practical generality by discovering online bin-packing heuristics that outperformed widely used baselines, with the evolved programs favoring tight-fit packing over the conventional best-fit strategy. The work was groundbreaking because it constituted the first demonstration that LLMs could contribute to genuinely novel mathematical discoveries, rather than merely recombining known results.

AlphaEvolve: From Functions to Full Programs

AlphaEvolve, unveiled by Google DeepMind in May 2025 [Novikov2025], dramatically expanded the scope of LLM-guided evolutionary search by evolving *entire codebases* rather than individual functions. The system employs an ensemble of Gemini models: Gemini Flash maximizes the breadth of candidate solutions explored per unit of compute, while Gemini Pro provides deeper, more insightful suggestions for particularly promising evolutionary trajectories. An evolutionary database determines which programs are selected for future prompts, and automated evaluators verify correctness and measure fitness across potentially long-running test suites executed in parallel across many machines.

The headline result was a breakthrough in matrix multiplication. AlphaEvolve discovered an algorithm for multiplying 4×4 complex-valued matrices using only 48 scalar multiplications, improving upon Strassen's 1969 algorithm, which required 49 multiplications in this setting. Because matrix multiplication algorithms compose recursively, even a single multiplication saved at the 4×4 base case compounds into substantial savings at higher dimensions. Applied to more than 50 open problems in mathematics, AlphaEvolve rediscovered the state-of-the-art solution for 75% of them and found strictly better solutions for 20%.

Beyond pure mathematics, AlphaEvolve produced results deployed in production at Google. It discovered a scheduling heuristic for Borg, Google's cluster workload management system, that has been running in production for over a year and recovers 0.7% of Google's overall compute capacity, allowing more tasks to execute on the same physical infrastructure. In hardware design, AlphaEvolve proposed modifications to Verilog code describing arithmetic circuits for matrix multiplication, eliminating redundancies that were subsequently integrated into a future Tensor Processing Unit design. It also optimized a matrix multiplication kernel used in Gemini model training, achieving a 23% speedup for that specific operation and reducing overall training time by approximately 1%. The breadth of these applications, spanning pure mathematics, systems software, hardware description languages, and machine learning infrastructure, underscores the generality that comes from evolving full programs rather than isolated functions.

Reflective and Adaptive Extensions

Several concurrent lines of work have sought to improve the sample efficiency and expressiveness of LLM-guided evolutionary search. ReEvo, introduced by Ye et al. and published at NeurIPS 2024 [Ye2024], frames the LLM as a *hyper-heuristic* and augments the evolutionary loop with a reflective mechanism. Each generation proceeds through selection, short-term reflection, crossover, long-term reflection, and elitist mutation. The reflection steps prompt the LLM to analyze why certain heuristics outperformed others, producing what the authors term "verbal

gradients" that steer subsequent generations toward more promising regions of the search space. Evaluated across six NP-hard combinatorial optimization problems, ReEvo produced state-of-the-art or competitive heuristics, evolutionary algorithms, and neural solvers while requiring fewer fitness evaluations than prior language-based hyper-heuristic methods.

EvoTune, proposed by Surina et al. and published at COLM 2025 [Surina2025], addresses a fundamental limitation of all prior systems: they treat the LLM as a *static* generator, discarding the rich signal produced by the evolutionary process itself. EvoTune closes this loop by alternating between evolutionary search and online reinforcement learning. During the search phase, the system operates in the standard FunSearch-like manner, sampling from a program database organized into islands and generating candidates via LLM inference. Periodically, the system switches to an RL fine-tuning phase in which the LLM is updated via online Direct Preference Optimization (DPO), leveraging pairwise comparisons between stored programs to teach the model to propose higher-quality mutations. Evaluated on bin packing, the traveling salesman problem, and the flatpack problem using small open-weight models (Llama 3.2 1B, Phi 3.5 Mini, and Granite 3.1 2B), EvoTune consistently outperformed the static-LLM baseline, demonstrating that even modest models can become effective search operators when fine-tuned on their own evolutionary trajectory.

Evolving Game-Theoretic Algorithms

The most recent extension of this paradigm applies AlphaEvolve to multi-agent reinforcement learning (MARL) algorithm design. Li et al. [Li2026] used AlphaEvolve's evolutionary framework to treat the source code of game-theoretic algorithms as a genome subject to LLM-driven semantic mutation. Rather than tuning hyperparameters or composing predefined building blocks, the system rewrites algorithmic logic, introduces new control flows, and injects novel symbolic operations. The search produced two previously unknown algorithm variants. Volatility-Adaptive Discounted CFR (VAD-CFR) dynamically adjusts its discounting parameters based on the volatility of regret updates, tracked via an exponential weighted moving average, and incorporates a hard warm-start that delays policy averaging until iteration 500. Smoothed Hybrid Optimistic Regret PSRO (SHOR-PSRO) mixes regret-based stability with aggressive greedy exploitation, annealing the balance over the training horizon. VAD-CFR matched or surpassed state-of-the-art performance in 10 out of 11 imperfect-information game environments, including Leduc Poker and Liar's Dice, with four-player Kuhn Poker as the sole exception. These results demonstrate that LLM-guided evolution can discover not only mathematical constructions and engineering heuristics but also fundamentally new algorithms in domains requiring strategic reasoning under uncertainty.

Synthesis

The trajectory from ELM through FunSearch, AlphaEvolve, ReEvo, EvoTune, and the MARL extension reveals a consistent pattern of increasing scope, autonomy, and integration. Each successive system has expanded the unit of evolution (from single functions to entire codebases), deepened the feedback loop (from static LLM queries to online fine-tuning), and broadened the domain of application (from mathematical curiosities to production infrastructure). The central insight unifying this line of work is that LLMs are most powerful not as oracles that produce answers, but as *search operators* whose generative capacity can be harnessed, evaluated, and iteratively refined within a principled optimization framework.

References

1. B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Brockman, T. Lattimore, Y. Li, et al., "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, pp. 468–475, Jan. 2024.
2. J. Lehman, J. Gordon, S. Jain, K. Ndousse, C. Yeh, and K. O. Stanley, "Evolution through large models," in *Handbook of Evolutionary Machine Learning*, W. Banzhaf, P. Machado, and M. Zhang, Eds. Singapore: Springer, 2024, pp. 331–366.
3. A. Novikov et al., "AlphaEvolve: A coding agent for scientific and algorithmic discovery," Google DeepMind, arXiv:2506.13131, Jun. 2025.
4. H. Ye, J. Wang, Z. Cao, F. Berto, C. Hua, H. Kim, J. Park, and G. Song, "ReEvo: Large language models as hyper-heuristics with reflective evolution," in *Proc. NeurIPS*, 2024.
5. A. Surina, A. Mansouri, L. Quaedvlieg, A. Seddas, M. Viazovska, E. Abbe, and C. Gulcehre, "Algorithm discovery with LLMs: Evolutionary search meets reinforcement learning," in *Proc. COLM*, 2025.
6. Z. Li, J. Schultz, D. Hennes, and M. Lanctot, "Discovering multiagent learning algorithms with large language models," arXiv:2602.16928, Feb. 2026.